SmartBIS 운영 수집 설계 (초안, GPS 없음) rev 15

> 상태: **초안**. 2026-10-31 기준. rev 14에 결정 4건(창 종료 5분, 방문당 상한 C안, TSB 통일, 7523 js·ud 7532 정정)을 반영했다. 처음부터 다시 작성했다. 표기: **[현재]** 확인된 현재 상태 · **[목표]** 구현·승인 후 상태 · **[결정]** 사용자 결정 완료 · **[결정대기]** 승인 필요 · **[미계산]** 값 미산출 · **[미확인]** 근거 확인 전 · **[추정]** 근거 부족

0. 용어 (Codex 참고)

- **route_id**: TAGO 노선 ID. 노선 번호가 아니다. 전체 105개.
- **node_order(순번)**: 한 route_id 안에서 기점을 0번으로 해 끝까지 1씩 증가하는 번호. 상·하행이 바뀌어도 0으로 돌아가지 않는다.
- **node_id**: 정류장 고유 ID. 이름이 같아도 node_id가 다르면 맞은편 정류장이다.
- **div**: 차량이 배정된 운행지역 코드. ud/ld = 관내 북/남, js/gj = 100번 장성발/광주발, hjs/hgj = 휴일 100번.
- **운행패턴**: div 안에서 차량들이 돌아가며 맡는 운행 묶음(예: ud1, ud2).
- **배차**: 특정 기간에 어느 차량이 어느 패턴을 맡는지 정한 것.
- **lane A/B/C**: 위치 API 호출을 세 갈래로 나눈 것. A = 장성100 상시, B = 출발 시각 기준 창, C = 전체 백업 순회.
- **창(window)**: lane B가 호출하는 시간 구간.
- **보간**: 중간 정류장 관측이 빠졌을 때 통과로 채우는 처리. 상태 추적과 학습에만 쓰고 운행 판정에는 쓰지 않는다.
- **방문당 상한**: 차량이 한 정류장에 접근하는 동안 도착정보 API를 부를 수 있는 횟수.
- **placeholder**: 실제 값이 없을 때 쓰는 임시값. 실측으로 취급하지 않는다.
- **9,500건 예산**: 하루 위치 호출 상한. 한도 10,000건에서 여유분을 뺀 값.

1. 전제

- 차량 단말에 GPS가 없다. 차량 위치는 TAGO 위치 API(`getRouteAcctoBusLcList`)로만 알 수 있다. 차량 상태는 수집 데이터로만 판단한다. **[현재]**
- 한도: API별 하루 10,000건(개발계정). 운영 예산 9,500건. 10,000건 한도의 공식 확인은 **[미확인]**.
- 운행 시간: 관내 06:15~21:00, 100번 05:30~. 100번(장성↔광주)은 평일 84회·휴일 74회.
- 노선 수는 기준별로 구분한다(혼용하지 않는다).
  - 운수사 관내 노선 **64개**(하루 편도 333회)
  - TAGO 관내 노선 ID **88개**(노선번호 기준 48개)
  - TAGO 100번 계열 ID **17개**(이름 기준 8개)
  - TAGO ID 합계 **105개** = 88 + 17
  - 운수사 64개와 TAGO 88개의 대응 관계는 **[미확인]**
- 운수사 노선과 TAGO ID를 정류장 순서로 맞춘 결과, 관내 운행의 69%는 TAGO ID 1개로 확정되고 31%는 후보가 2~7개다. 상세 대응표는 이 문서에 넣지 않는다.
- 위치 API는 **노선 단위**다. 1회 호출로 그 노선의 모든 차량(차량번호·위치)이 온다.
- 도착정보 API 응답에는 차량번호가 없다. 같은 노선·방향에 차량이 여러 대면 구분이 어렵다.
- TAGO 데이터 갱신 주기는 10~20초로 본다. **[미확인]**, 실측 필요.
- 정류장 간 평균 약 1.2분은 **[추정]**이며 placeholder로만 쓴다.

2. 차량 배정 자료 (CSV 3개)

CSV 원문은 이 문서에 표로 넣지 않는다. 행 수가 많아 md 변환이 실패했다. Codex는 아래 경로의 원본 CSV를 직접 읽는다.

2.1 파일 위치

- 기본 폴더: `D:\PROJECT\SmartBIS_jangsung\노선 및 운행 관련 자료\군민운수 운행관리시스템\`
- 세 파일은 역할이 다르다. 각 파일의 설명을 섞지 않는다.

2.2 차량구분.csv (차량 → 운행지역)

- 파일: `차량구분.csv`
- 열: `vehicle_no`(차량번호), `div`(운행지역), `description`(영문 설명)
- 역할: **누가 어느 지역**에 배정되는지. 지역 안에서 무엇을 운행하는지는 담지 않는다.
- 구성: js 12대(7536 포함), gj 2대, ld 8대, ud 10대, rb 3대(예비). 합계 33대.
- **7523**: `js`로 확정한다(**[결정]**). ud에 있던 7523 행은 오타로 보고 **7532**로 정정한다(**[결정]**). 정정 후 ud는 10대다. 7532는 현 CSV 원문에 없으므로 원본 수정 시 반영 여부를 Codex가 확인한다.
- rb(예비)는 운행지역이 없는 예비 차량이다.
- **7536**: 2026-10-31부터 운행을 시작한다. 장성터미널↔광주첨단제일풍경채 노선(첨단100)에 고정 투입한다. 첨단100은 TAGO 카탈로그에 대응 ID가 없다. 승인 후 div js10을 생성할 예정이다(**[결정대기]** 승인 상태 확인).

2.3 운행패턴.csv (지역 안의 운행 묶음)

- 파일: `운행패턴.csv`
- 역할: **지역 안에서 무엇을 어떤 순서로** 운행하는지. 누가 맡는지는 담지 않는다.
- 구조: 한 행 = 한 패턴(div + 패턴명). 이후 열은 운행 순서별로 (순번, 시각, TAGO 노선 ID) 세 칸씩 묶여 있다.
- 예: `ud1`은 14회 운행하며 첫 운행은 06:20, 노선 ID는 TBS342000033이다. 패턴마다 운행 횟수가 다르다(예: ud3 18회, ld1 8회).
- **노선 ID 접두어**: 관내 행은 `TBS342…`, 100번 행은 `TSB342…`로 적혀 있다. **[결정]** 같은 노선으로 보고 `TSB342…`로 정규화한다. 정규화 후에도 TAGO 카탈로그에 없는 ID는 별도로 표시한다.
- **7536 노선 2개**(TAGO 카탈로그에 없음, routeid 없음):
  - 노선 1: 상행 장성터미널→남면→나노산단→제일풍경채 / 하행 제일풍경채→나노산단→삼태→남면→장성터미널
  - 노선 2: 상행 장성터미널→진원면→고산→작동→힐스테이트 / 하행 힐스테이트→작동→진원면→장성터미널

2.4 배차.csv (배정 규칙의 예시)

- 파일: `배차.csv`
- 역할: **언제 누가 무엇을** 맡는지의 예시. 규칙 설명용이다.
- 규칙 **[현재]**:
  - 관내(ud·ld): 매주 패턴이 바뀐다. 예: 00차량이 이번 주 ud1이면 다음 주는 ud2다.
  - 100번(js·gj): 매일 패턴이 바뀐다.
  - 휴일 100번(hjs·hgj): 휴일 안에서 매일 바뀐다.
- 배차.csv를 실제 판정 입력으로 쓸지는 Codex가 파일을 확인한 뒤 **[결정대기]**로 둔다.

2.5 파일명 확인

- 같은 폴더에 관내·광주를 나눈 파일명(관내 운행 패턴.csv, 광주 운행 패턴.csv, 관내배차.csv, 광주배차.csv)이 있다는 기록이 있다. **[미확인]** Codex는 폴더에서 실제 파일명을 확인하고, 위 2.2~2.4와 같은 자료인지 판정한다. 다르면 이 장을 정정한다.

2.6 차량 변경 관리

- 차량 변경(수리 등)은 1회성이거나 수리 기간 동안만 이어진다고 예상한다. **[추정]**
- 수집 데이터에서 차량과 노선이 계속 맞지 않으면 "변경 후보"로 기록한다.
- 장기화 기준(기간·횟수)은 **[결정대기]**다.
- 기준을 넘으면 대시보드에 "관리자 확인 대기"로 표시한다. **[목표]** 관리자가 확인하기 전에는 배차 정보를 자동으로 바꾸지 않는다.

3. 차량 상태와 정류장 관측 보간

상태	뜻	호출
SEARCH	단말은 켜졌으나 노선을 모름	lane C 스윕 대상
TRACK	노선을 알고 있음(순번을 알면 이후 운행도 예측)	운행 중인 노선을 주기적으로 조회
LAYOVER	운행을 마치고 대기 중	다음 출발 2분 전까지 호출 없음. 순번을 모르면 직전 노선만 감시(최대 30분, 설계 값)

- **통과 판정**: 응답에는 정류장 통과 시각이 없다. `observed_at`은 저장 시각이다. 통과는 `node_order`가 증가한 것으로만 판정한다. **[현재]**
- **보간 규칙**: 같은 `route_id` 안에서 연속된 두 관측의 `node_order`가 증가하고, 그 사이 누락 정류장이 **5개 이하이며 30분 이하**이면 통과로 보고 추정 시각을 넣는다(`interpolated=true`).
  - 누락 6개 이상 또는 30분 초과 → "구간 미확인".
  - 순번이 감소하거나 `route_id`가 바뀌면 보간하지 않는다.
- 보간 행은 상태 추적과 학습에만 쓴다. 운행 판정(일치·불일치·미관측)에는 실제 관측만 쓴다.

4. 위치 API 호출 (①위치 수집 프로세스만)

4.1 공통

- **DB 적용**: backend는 Flyway를 쓰지 않는다. `server/database/migrations`를 V001→V019 순서로 `psql`에 적용한다. **[현재]**
- **적용 범위**: ①위치 수집만 TAGO 위치 API를 호출한다. ⑤앱 API와 ⑥화면은 DB만 읽는다. **[결정]**
- **호출 시간대**: 05:15~21:15에만 호출한다. 21:15~05:15에 호출 시도가 생기면 로그만 남긴다(코드 고정). **[결정]**
- **예산 가드**: 하루 9,500건. 재시도 최대 2회, 재시도 간격 10초. 실패한 시도도 예산에 포함한다. 사용량은 `guard_usage_daily`에 기록하고 재시작 시 당일 값을 복원한다. 상한에 도달하면 그날 위치 호출을 멈추고 로그를 남긴다.
- **lane 분리**: 세 lane은 각자 스레드·간격·throttle을 가진다. 도착정보 호출과 throttle을 공유하지 않는다. lane별 throttle 분리 여부는 **[현재] 미확인**(코드 확인 필요).

4.2 Lane A (장성100 상시)

- 대상 ID 목록은 **[결정대기]**다. 확정 전에는 대상이 0개다.
- 확정되면 05:15~21:15 동안 10분 주기로 조회한다.

4.3 Lane B (출발 시각 창)

- **기준 출발 시각**: 운수사 시간표의 기점 또는 종점 출발 시각.
- **창 시작** = 기준 출발 − 15분(관내), − 10분(변형 100번). **[결정]**
  - 예: 118번 기준 출발 07:15 → 창 시작 07:05(변형 100번).

- **창 종료** = 그 차량이 **노선의 마지막 순번(`node_order`)을 관측한 시점**. **[결정]**
  - 편도 소요 시간으로 창 끝을 계산하지 않는다. 창 계산에 편도 소요 시간을 쓰지 않는다.
  - **상한 5분 [결정]**: 마지막 순번을 관측하면 그 시점부터 5분 뒤에 창을 닫는다. 마지막 순번을 끝내 관측하지 못하면 같은 5분 기준으로 강제 종료한다(**[추정]**, 강제 종료 기준 확인 필요).
  - 정책 키: `location.laneB.close_after_last_min = 5`.
- 창 안에서만 10분 주기로 호출하고, 창 밖에서는 호출하지 않는다. **[현재 설계]**
- 같은 ID의 창이 겹치면 하나로 합친다.
- 창 계산에 쓰는 값은 정제된 데이터만 쓴다(9장). 학습값은 도착 추정(5장)에만 쓴다.

4.4 Lane C (백업 스윕)

- 105개 ID 전체를 06:00~21:00 동안 30분마다 1회전한다.
- 1회전은 ID 사이 2초 간격의 순차 호출이다(105 × 2초 = 210초, 약 3.5분). 1회전이 끝나면 다음 주기까지 쉰다.
- 429 백오프가 걸리면 1회전 시간이 늘어난다.

4.5 429 대응

- 429가 나오면 lane C 간격을 2초 → 3초 → 5초로 단계적으로 올리고 로그를 남긴다. 단계값은 설정값이다.

4.6 시간표 없는 ID

- 시간표가 없는 ID는 lane C만 적용한다.
- 장성100 ID는 lane B에 넣지 않는다.
- 이름 미확인 ID 8개(104·106·108·109·110·111·112·154)는 lane C만 적용한다.

4.7 판정 분담

- 스윕과 창의 관측 결과는 ③이 전날 밤에 해석한다.
- ①은 판정 결과인 `route_call_plan`만 읽으며 판정 로직을 알지 못한다.

4.8 관측 모드

- 8장을 따른다. 105개 10초 순환은 폐기한다.

5. 도착정보 API

5.1 목적과 규모

- 정류장에서 "다음 차량이 언제 오는지"를 보여준다. 정류장마다 계속 조회하면 한도를 넘는다.
- 관내+100번 하루 약 13,000~14,000건으로 추정했으나 **[미계산]**.

5.2 조회 시점 (3가지) **[현재 설계]**

1. 운행을 시작할 때
2. 다음 정류장까지 예상 소요가 3분 이상일 때
3. 예정 시각이 지났는데 아직 통과하지 않았을 때

5.3 방문당 상한 — **C안 [결정]**

- **용어**: "방문"은 차량이 한 정류장에 접근하는 한 번의 과정이다.
- **규칙**:

    - **예상 소요 3분 미만**: 도착정보 API를 방문당 **최대 2회** 호출한다.
    - **예상 소요 3분 이상(긴 구간)**: 도착정보 API를 **호출하지 않는다.** 학습된 정류장 간 소요 시간을 표시한다. 학습값이 없으면 `정류장 수 × 1.2분`(placeholder)을 쓴다.
- 이 상한은 코드 고정이며 정책 키가 아니다. 설정으로 바꾸지 않는다.
- 현재 코드 `ArrivalVisitStore.java:32`(`query_count < 2`)는 C안과 맞다. DB CHECK 여부는 **[미확인]**.
- 긴 구간의 위치 추적은 lane B·C의 위치 API 값으로 한다.

5.4 짧은 구간 계산

- 쌓인 위치 기록에서 학습한 정류장 간 소요 시간(시간대별 중앙값)으로 계산한다. **[목표]**
- 학습 전에는 "마지막 도착 예정 시각 − 경과 시간"을 보여준다. **[현재 설계]**

5.5 응답 매칭

- 응답의 `arrprevstationcnt`(남은 정류장 수)가 차량의 현재 위치와 맞는 값을 고른다. **[미확인]**, 실측으로 확인한다.

5.6 관내 혼동

- 관내 노선은 같은 노선·방향에 우리 차량이 1대인 경우가 많아 혼동이 적다. **[추정]**

5.7 2분 위치 기준 폐기 **[결정: 폐기 / 대체 조건 결정대기]**

- 현재 `ArrivalService.java:96,112`에 "최근 2분 이내 위치 행"을 요구하는 조건이 있다. 설계 근거가 없으므로 폐기한다.
- 위치 스위치가 꺼져 있으면 도착이 0건이 되는 것은 코드 경로상의 예상이며 실측이 아니다. **[현재]**
- **대체 조건 [결정대기]**: 검토용 제안은 "남은 정류장 수가 1~2개이고 예상 3분 이하일 때 조회"다. 실측 전이므로 확정하지 않는다.

5.8 보간 행과 수집률

- 보간 행은 학습에만 쓰고 운행 판정에는 쓰지 않는다. **[결정]**
- 수집률(관측 정류장 수 ÷ 노선 정류장 수)은 요약 지표로만 쓴다. 노선 제외 기준이 아니다. **[결정]**

5.9 도착 스위치

- `TAGO_ARRIVAL_ENABLED`는 **[현재] false**다.
- 위치 수집을 재개할 때 함께 `true`로 켠다. **[결정]**, 승인 전.

6. 예산 추정 **[미계산]**

- Lane C: 105 × 30회 = **3,150건/일**(재시도 제외). **[확정]**
- Lane A: N × 96회/일. N은 장성100 대상 ID 수(**[결정대기]**).
- Lane B: 창 길이가 "마지막 순번 관측 + 5분"으로 바뀌었으므로 계산 방법을 다시 정해야 한다. **[미계산]**
- 도착 API 호출량: **[미계산]**.
- 재시도·429 여유: **[미계산]**.
- 합계가 9,500건 이하인지 확인하기 전에는 위치 호출 재개를 승인하지 않는다. **[결정]**

7. 한도를 넘을 때의 선택지 (효과가 큰 순)

0. **lane C 주기 연장**: 30분 → 더 긴 주기(`location.laneC.cycle_min`을 먼저 조정).
1. **정밀도 낮추기**: 2정류장마다 확인하고, 중간은 학습된 소요 시간으로 추정한다. 호출은 약 절반이 되지만 정확도가 떨어진다.
2. **도착정보로 통과 판정**: 같은 노선·방향에 1대만 운행할 때만 쓸 수 있다. 관내 노선 대부분이 여기에 가깝다(하루 10회 이하 노선 89%, 추정). 위치 호출은 줄고 도착 호출은 늘어난다.

8. 관측 모드 (먼저 할 일)

- **기간**: 평일 3일 + 토요일·일요일. **[결정]**
- **호출 규칙**: 4장의 lane A·B·C를 그대로 쓴다. 관측과 운영의 호출 규칙이 같아야 결과를 운영에 쓸 수 있다. **[결정]**
- **저장**: 응답에 나온 **모든 차량**(차량번호, route_id, node_order, 시각)을 저장한다. 관측 모드가 전체 차량 저장 플래그 역할을 한다. 저장 동작은 **[미확인]**, Codex가 확인한다.
- **얻는 값**:
  - 순번 운행표와 실제 운행의 차이(지연·결행·순번 교체)
  - 후보가 여러 개인 16개 노선(관내 운행 103회)이 실제로 어느 TAGO ID로 잡히는지, 대응표에 없는 TAGO ID 6개에 차량이 나타나는지
  - 창 밖에서 발견된 차량 건수(창 규칙 검증)
  - 기준 출발 시각과 실제 첫 관측 시각의 차이(창 시작점 검증)
  - 정류장 간 실제 소요 시간(시간대별, 정제 후)
  - 차량별 실제 운행 시간의 합 = 차량별 (첫 관측 ~ 마지막 관측) 시간의 합
  - 같은 노선·방향을 동시에 달리는 비율
- 관측 모드 기동은 로직·테스트·기본값 확인 후에만 승인한다. **[결정]**

9. 기존 코드와의 관계 (현재 vs 목표)

항목	[현재]	[목표]
순번 0 허용	**완료**: 로직 `18cf33f`, Catalog `8835d90` (로컬 커밋, push 안 함)	유지
위치 API 순번 0	기점 0 허용 코드 있음. 실제 응답의 기점 순번은 **[미확인]**	응답으로 확인
lane 창 방식	`18cf33f` 커밋, LocationLaneScheduleTest 8건 통과. 창 종료는 **[현재] 편도 소요 방식이 아님, 마지막 순번 관측 + 5분으로 변경 필요**	창 종료를 "마지막 순번 관측 + 5분"으로 변경(4.3). 코드·테스트 수정 필요
도착 방문당 상한	2회(`ArrivalVisitStore.java:32`)	C안과 일치. 긴 구간 호출 차단 로직 확인 필요
2분 위치 기준	코드에 존재(`ArrivalService.java:96,112`)	폐기. 대체는 5.7 결정 후
방향 조회	UP/DOWN 조건 조회. `updowncd=2` 노선이 404가 될 수 있음(**[추정]**, 재현 미확인)	방향 조건 제거, `route_id` + `node_order + 1`로 다음 정류장 찾기
throttle	lane별 분리 여부 **[미확인]**	lane별 분리
TagoQuotaGuard	예산 가드 유지	lane마다 `window`, `min-gap-ms` 설정
정책 조회	미구현	11장, 단계 1

- 테스트 단계(단말 3대, 10초 간격)와 운영의 차이는 설정값과 상태 판단으로만 둔다. 테스트 전용 분기 코드는 만들지 않는다. **[결정]**
- 편도 소요 학습값은 **정제 조건을 통과한 데이터만** 쓴다. 전환 중앙값 49.76초(p90 약 2,164초)는 날짜·회차 미분리, 순번 감소 774건, 건너뜀 2,551건이 섞인 집계이므로 **인접 정류장 통행 시간으로 쓰지 않는다. [결정]**

10. 노선 자료 (TAGO 105개)

- 구조: 왕복형 74개 ID, 편도형 31개 ID. 합계 105. **[현재]**
- 순번은 route_id 안에서 0부터 끝까지 이어진다. 상·하행이 바뀌어도 다시 시작하지 않는다.
- ID 접두어: 관내 `TBS342...`는 `TSB342...`로 정규화한다(2.3). **[결정]**
- **장성100 대상 ID(N)**: **[결정대기]**. 102·103은 장성100으로 보고 lane B에서 제외한다(T7).
- **확정 변형 100번 ID 목록**: **[결정대기]**. 확정 전에는 lane B 대상에 넣지 않는다.
- **이름 미확인 ID 8개**: lane C만 적용(4.6).

11. 설정 분리

11.1 원칙

- **배포 설정(`.env`, `compose.yaml`)**: 서버마다 다른 값과 비밀값만 둔다. DB 접속, 포트, 인증서 경로, 로그 경로, 타임존, `TAGO_SERVICE_KEY`.
- **기능 스위치(비상 차단용)**: `TAGO_ENABLED`, `TAGO_VEHICLE_LOCATION_ENABLED`, `TAGO_ARRIVAL_ENABLED`. 기본값은 안전 쪽(`false`)이다. 스위치가 `false`이면 정책 값과 무관하게 해당 기능은 호출하지 않는다.
- **운영 정책(정책 테이블, 애플리케이션 관리) [목표]**: 호출 시간대, lane별 주기, 스윕 주기, 429 백오프 단계, 창 상한 분, 재시도 횟수·간격, 관측 모드, 카탈로그 동기화 주기. 재기동 없이 바꾼다.
- **노선 자료(시간표 테이블)**: 노선별 기준 출발 시각, 편도 소요 학습값, lane A·B 대상 ID, 노선 구조. 운수사 자료 등록으로만 바꾼다.
- **코드 고정(정책 아님)**: 도착 방문당 상한(3분 미만 2회, 긴 구간 호출 없음), 21:15~05:15 위치 호출 금지, 순번 0 기준, 상태 전이 규칙, 일일 예산 하드 상한 검사.

11.2 우선순위와 조회 방식 **[목표]**

- 우선순위: **노선 값 > 서비스 값 > 프로필 기본값 > 코드 안전 기본값**.
- 정책 테이블 `tago_policy`: `policy_key`, `value`, `value_type`, `scope_type`(GLOBAL/SERVICE/ROUTE), `scope_id`, `profile_name`, `updated_by`, `updated_at`, `note`.
- 변경 이력 `tago_policy_history`: 이전 값, 새 값, 변경자, 시각을 남긴다.
- 반영: 60초마다 재로딩하거나 관리 API 호출 시 즉시 반영한다.
- 조회 실패 시 마지막 정상 값을 쓴다. 정상 값이 없으면 **위치 수집을 끈 상태**로 둔다.
- 정책 값이 코드 고정 규칙을 어기면 저장을 거부하고 로그를 남긴다.

11.3 정책 키 (초기값)

키	초기값	설명
`location.laneA.enabled`	true	장성100 상시 수집(대상 확정 전에는 대상 0)
`location.laneA.interval_min`	10	lane A 주기(분)
`location.laneA.window`	05:15~21:15	lane A 호출 시간대
`location.laneB.lead_min.regular`	15	관내 창 시작 = 기준 출발 − 15분

키	초기값	설명
`location.laneB.lead_min.variant`	10	변형 100번 창 시작 = 기준 출발 − 10분
`location.laneB.close_after_last_min`	**5**	마지막 순번 관측 후 창을 닫기까지 시간(분). 미관측 시 강제 종료에도 적용(가정, 확인 필요)
`location.laneB.interval_min`	10	lane B 주기(분)
`location.laneC.enabled`	true	백업 스윕
`location.laneC.window`	06:00~21:00	스윕 시작 가능 시간대
`location.laneC.cycle_min`	30	스윕 주기(분). 예산 부족 시 먼저 늘린다
`location.laneC.gap_sec`	2	ID 간 간격(초), 1~3초 범위
`location.laneC.backoff_429_sec`	2,3,5	429 시 단계 간격
`budget.daily_limit`	9500	일일 위치 호출 상한(계정 제약, 변경은 승인 후)
`budget.retry_max`	2	재시도 횟수
`budget.retry_gap_sec`	10	재시도 간격
`observation.mode`	false	전체 차량 저장 플래그. 호출 규칙은 4장과 동일. 운영에서는 false 고정
`catalog.sync_cron`	`-`(차단)	**현재 운영 값**. 재개 값은 승인 후 결정
`arrival.enabled`	false	**현재 운영 값**. 위치 재개 시 함께 true(결정, 승인 전)

- 코드 고정(정책 키 아님): 위치 호출 금지 구간 21:15~05:15, 도착 방문당 상한, 순번 0 기준.

11.4 프로필 **[목표]**

- 프로필 4종: 대도시형, 중소도시형, 도농복합형, 농촌형. 프로필은 정책 키의 기본값 묶음이다.
- 장성군은 도농복합형 후보다. 노선 자료로 확정하기 전까지는 11.3 초기값을 쓴다.
- 서비스 구분: **차량 서비스**(위치 수집, 창 계산), **정류소 서비스**(도착정보), **공공시설 서비스**(노선과 무관한 조회).

11.5 단계

- **단계 0(비상 차단, 현재 상태)**: 위치 스위치 `false`, 관측 모드 `false`, 카탈로그 동기화 `-`, 도착 스위치 `false`. 운영 backend 재시작 정책 `no`. **[현재]**
- **단계 1(소스 수정, 아침 기동 전)**: 11.2~11.3 정책 조회 로직 추가. 4장 호출 규칙이 정책 값을 읽도록 바꾼다. 창 종료 규칙(4.3)과 방문당 상한(5.3)을 반영한다. 테스트는 프로필별 값, 우선순위, 조회 실패 시 안전 기본값, 시각 고정 호출 규칙을 검증한다. 실제 TAGO 호출은 하지 않는다. 빌드·배포는 별도 승인.
- **단계 2(낮 운영 안정화 후)**: 관리 API와 화면, 변경 이력 조회, 프로필 선택.
- **[결정대기]** 정책 테이블(V020)을 단계 1에 포함할지. 기본안은 단계 1에서 별도 적용하며, 적용 전에 백업을 다시 받는다.

11.6 위치 수집 재개 시 `.env` 변경 항목 (승인 대상)

- `TAGO_VEHICLE_LOCATION_ENABLED`: 승인 후 `true`. 조건은 T9 통과, 6장 예산 계산 완료, 승인.

- `TAGO_ARRIVAL_ENABLED`: 위치 스위치와 **함께** `true`. **[결정]**, 승인 시 적용.
- `TAGO_LOCATION_OBSERVATION_MODE`: `false` 유지.
- `TAGO_CATALOG_SYNC_CRON`: `-` 유지(재개 방식 결정 전).
- `TAGO_ENABLED`: `true` 유지.

12. 검증 기준

12.1 노선 유형별 기점·종점 기대값

- **왕복형(74개 ID)**: 한 ID 안에서 기점(UP 0번)과 종점(DOWN 마지막 순번)이 **같은 정류장 node_id**다. 예: TSB342000143, UP 757=0, DOWN 757=76.
- **편도형(31개 ID)**: `updowncd=2`. 한 ID는 한 방향이고 순번은 0부터 끝까지 이어진다. 기점과 종점은 서로 다른 정류장이다.
- **공통**: 순번은 route_id 안에서 0부터 끝까지 이어진다.

12.2 연속성 판정 (노선 단위)

- `min(node_order) = 0`
- `count(*) = count(DISTINCT node_order)` (중복 없음)
- `max(node_order) - min(node_order) + 1 = count(*)` (빠짐 없음)
- 세 조건을 모두 만족하면 통과. 샘플 노선(`SAMPLE_ROUTE_001`)은 검사하지 않는다. 판정은 순번만 본다.

12.3 기점 이름 대조

- 기준: 엑셀 `TAGO_노선도.xlsx`의 노선별 기점·종점 이름과 DB의 `node_id`.
- 조인: `bus_route_stops.node_id` = `bus_stops.node_id`. 이름은 `bus_stops.node_name`.
- 판정: 0번 이름과 node_id가 엑셀 기점과 같으면 통과. `node_name`이 NULL이면 **미확인**.
- 1차 대상: 왕복형 대표 ID(0001, 0143), 편도형 대표 ID(0002, 0044, 0105, 0111).

12.4 0번 기준 도착정보 확인

- 대상: 0번 정류장 중 하루 운행이 확인된 노선 2개 이내.
- 방법: 도착 API를 1~2회만 호출한다(방문당 상한 2회). `arrprevstationcnt`가 차량 위치와 맞는지 본다.
- 실제 TAGO 호출은 **운영 승인 후 낮 시간**에만 한다. 개발 테스트에서는 호출하지 않는다.

12.5 테스트 목록

번호	테스트	통과 기준	현재
T1	창 안·밖 호출	창 안 1건, 창 밖 0건	통과(18cf33f). **창 종료 규칙 변경 후 재작성 필요**
T2	21:15~05:15 호출 금지	위치 호출 0건, 로그만	통과(18cf33f)
T3	창 겹침 병합	겹치는 창이 하나로 합쳐짐	통과(18cf33f). 창 종료 변경 후 재확인
T4	lane C 1회전	105건, ID 간격 2초	통과(18cf33f)
T5	429 백오프	2→3→5초, 로그 기록	통과(18cf33f)

번호	테스트	통과 기준	현재
T6	창 시작 계산	118 기준 07:15 → 시작 07:05(변형 −10분). 153 기준 06:45 → 시작 06:35(153 변형 여부 **미확인**)	통과(18cf33f)
T7	장성100 제외	102·103이 lane B 대상이 아님	통과(18cf33f)
T8	0번 기준 입력	0143 UP 757 순번 0, DOWN 757 순번 76	통과(`8835d90`)
T9	빌드·테스트 실행	exit 코드, Tests run / Failures / Errors, 실패 0	**미통과**: 빌드 통과, tago 103건 중 `RouteCallPlanTest` 1건 실패
T10	창 종료 5분 규칙	마지막 순번 관측 후 5분에 창 닫힘, 5분 뒤 호출 0건	**신규, 미구현**
T11	방문당 C안	예상 3분 미만 도착 호출 최대 2회, 3분 이상 도착 호출 0회	**신규, 미구현**
T12	TBS→TSB 정규화	`TBS342000033`이 `TSB342000033`으로 매칭	**신규, 미구현**

- T9 실패 원인: 테스트 기대가 오래됐을 가능성이 높다(**[추정]**). 구현 경로(`RouteCallSchedule.java:38`)에서 Lane A 비활성 계획의 `service=null`은 의도된 경로로 보인다. 정정안은 아직 적용하지 않았다.
- 테스트 실행 기록은 로그 파일로 저장하고, 보고에는 tail 발췌와 숫자만 쓴다.

13. 작업 범위와 현재 상태

13.1 완료

- lane 창 방식(이전 기준), 21:15~05:15 호출 금지, 429 백오프, 0번 순번 하한 → 로컬 커밋 `18cf33f`, `8835d90`. push 안 함.
- 운영 backend 재시작 정책 `no`(compose.yaml 변경, 커밋 전).
- T1~T8 통과(T8은 `8835d90`).

13.2 다음 구현 후보 (각 항목 승인 후)

- (a) 창 종료를 "마지막 순번 관측 + 5분"으로 변경하고 T1·T3·T6 재작성, T10 추가
- (b) 방문당 C안 적용, T11 추가
- (c) 2분 기준 폐기와 대체 조건 적용 (5.7 결정 후)
- (d) RouteCallPlanTest 기대 정정 → T9 재판정 (승인 필요)
- (e) 정책 조회 로직(11장, 단계 1)
- (f) TBS→TSB 정규화, T12 추가

13.3 범위 밖 (실행하지 않음)

- 운영 이미지 교체(`latest` → `head-8835d90`), 위치·도착 재개, 관측 모드 기동, 측정 컨테이너 기동, push.
- 12장 검증 실행(노선 기점 대조 전수, 도착 0번 확인).

13.4 확정되지 않은 입력과 기본값

- 장성100 대상 ID(N): 확정 전 lane A 대상 0.
- 확정 변형 100번 ID: 확정 전 lane B 대상 제외.
- 이름 미확인 ID 8개: lane C만.
- 편도 소요 학습값: 창 계산에 쓰지 않음. 도착 추정에만 쓰며, 값이 없으면 정류장 수 × 1.2분(placeholder).
- 관내 창 건수 2,058: 임시값(재계산 필요).
- 강제 종료 기준(미관측 시): 5분 적용을 가정. 확인 필요.

13.5 체크리스트

- 로직 수정 범위 확정: ☑
- 0번 순번 하한 수정 및 T8 통과: ☑
- lane 창 방식 T1~T7 통과(이전 기준): ☑
- 창 종료 5분 규칙 반영 및 T1·T3·T6 재작성, T10 추가: ☐
- 방문당 C안 반영, T11 추가: ☐
- 2분 기준 대체 조건 결정(5.7): ☐
- TBS→TSB 정규화, T12 추가: ☐
- 2장 CSV 파일명·위치 확인(2.5): ☐
- 7523·7532 정정 원본 확인: ☐
- T9 전체 통과(실패 0): ☐
- 예산 계산(6장): ☐
- 정책 조회(11장) 별도 승인: ☐
- 검증 실행(12장) 별도 승인: ☐
- 위치 수집 재개 승인: ☐
- 도착 스위치 재개 승인(위치와 함께): ☐

14. 결정·확인 필요 (남은 것)

1. 장성100 대상 ID 목록(N) 확정
2. 확정 변형 100번 ID 목록 확정
3. 이름 미확인 ID 8개(104·106·108~112·154)의 노선 확인
4. 차량 변경 장기화 판단 기준(기간·횟수)
5. 2분 위치 기준 폐기 후 대체 조건(채택 여부)
6. 정책 테이블(V020)을 단계 1에 포함할지 여부
7. 예산 계산 완료와 10,000건 한도의 공식 확인
8. 운수사 64개와 TAGO 88개의 대응 관계 확인
9. RouteCallPlanTest 기대 정정 승인
10. 위치 재개 시각과 방식
11. 7536의 운행 시작(2026-10-31)과 첨단100 승인, js10 생성
12. 7532 정정의 원본 CSV 반영 확인(ud 행)
13. 배차.csv를 실제 판정 입력으로 쓸지 여부
14. 위치 재개 시 도착 스위치(`TAGO_ARRIVAL_ENABLED`)를 함께 켜는 결정의 승인
15. 미관측 시 강제 종료에도 5분을 적용할지 확인